CodedSoul_Notes

"Code today, Solve tomorrow."

C#

INTERVIEW Q&A
+ REAL WORLD CONCEPTS
NOTES

Concepts • Questions • Revision

Core Concepts

Practice Q&A

Quick Revision

Interview Focused

OOP & Design

Collections & LINQ

Async, Await & Modern C#

Real World Scenarios

Best Practices & Patterns



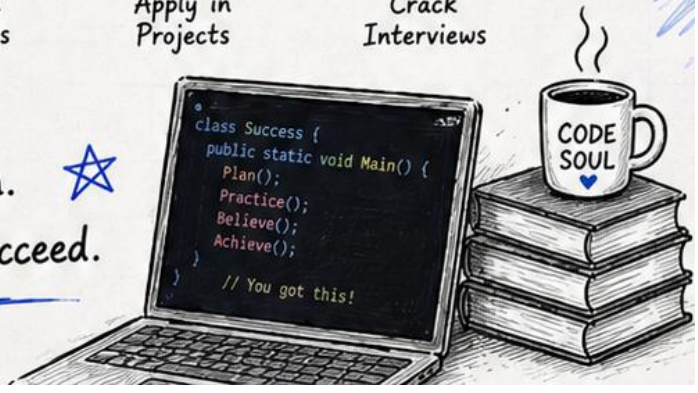
> "It's not about memorizing answers,
> It's about understanding deeply."

C# Success Pipeline

Learn Concepts → Understand Deeply → Practice Questions → Apply in Projects → Crack Interviews

- ✓ Clear Concepts
- ✓ Real World Applications
- ✓ Smart Practice
- ✓ Confident Answers
- ✓ Interview Ready!

Think. Learn.
Implement. Succeed.



1. C# Introduction & Setup

- **What is C#?**

  C# (pronounced "C sharp") is a modern, object-oriented, type-safe programming language developed by Microsoft as part of the .NET framework.

- **Why C#?**
  - Simple, modern and easy to learn
  - Object-oriented and feature rich
  - Strongly typed and secure
  - Platform independent with .NET
  - Used in Web, Desktop, Mobile, Games, Cloud & more
  - Backed by Microsoft with huge community support

- **Structure of a C# Program**

Using Directives	→	Import namespaces
Namespace Declaration	→	Organize code
Class Declaration	→	Blueprint of objects

Main() Method
{
Statements;
return 0;
}
	→	Program execution starts from Main()

Other Methods / Functions	→	Reusable code blocks

- **Basic Syntax of C# Program**

```
using System;                    // Using directive
namespace MyFirstProgram         // Namespace
{
  class Program                  // Class
  {
    static void Main(string[] args) // Entry point
    {
      Console.WriteLine("Hello, World!"); // Statement
    }
  }
}
```

Note :
Every C# program must have a Main() method.

2. Variables, Data Types & Operators

Variables in C#

A variable is a named storage location used to store data that can be changed during program execution.

Syntax :

```
dataType variableName = value;
```

Example :

```
int age = 20;
string name = "Nikhil";
```

Note :
In C#, every variable must be declared before it is used. ☺

Data Types in C#

Category	Type	Description	Example
Value Types	int	Stores whole numbers	int x = 10;
	float	Stores decimal numbers	float f = 5.6f;
	char	Stores a single character	char ch = 'A';
	bool	Stores true or false	bool isActive = true;
Reference Types	string	Stores text	string name = "Hello";
	array	Stores a collection of elements	int[] arr = {1, 2, 3};
	class	User-defined data type	Student s = new Student();
Other Types	decimal	High precision decimal values	decimal d = 12.45m;
	var	Implicitly typed (type inferred)	var a = 100;

Operators in C#

1. Arithmetic

+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus
++	Increment
--	Decrement

2. Relational

==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater or equal
<=	Less or equal

3. Logical

&&	Logical AND
||	Logical OR
!	Logical NOT

4. Assignment

=	Assign
+=	Add & assign
-=	Subtract & assign
*=	Multiply & assign
/=	Divide & assign
%=	Modulus & assign

5. Other

&	Bitwise AND
|	Bitwise OR
^	Bitwise XOR
~	Bitwise NOT
<<	Left shift
>>	Right shift
?:	Conditional (Ternary)

Example Program

```
using System;
class Program
{
    static void Main(string[] args)
    {
        int a = 10, b = 5;
        int sum = a + b;    // Addition
        bool isGreater = a > b; // Relational
        Console.WriteLine($"Sum = {sum}, Is a > b ? {isGreater}");
    }
}
```

Key Points :

- ✓ Variables store data.
- ✓ Data types define the kind of data a variable can store.
- ✓ Operators perform operations on variables and values.
- ✓ Strong typing makes C# safe and reliable. ☆

3. Control Statements

What are Control Statements?

Control statements are used to control the flow of execution in a program based on certain conditions or loops.

Note :
They help the program to make decisions and repeat actions.

1. Selection Statements (Decision Making)

1. if Statement

```
if (condition)
{
    // block of code
}
```

2. if...else Statement

```
if (condition)
{
    // block of code
}
else
{
    // else block
}
```

3. else if Ladder

```
if (condition1)
{
    // block
}
else if (condition2)
{
    // block
}
else
{
    // else block
}
```

4. switch Statement

```
switch (expression)
{
   case value1:
        // block
        break;
   case value2:
        // block
        break;
   default:
        // block
        break;
}
```

2. Iteration Statements (Loops)

1. while Loop

```
while (condition)
{
    // block of code
}
```

2. do...while Loop

```
do
{
    // block of code
}
while (condition);
```

3. for Loop

```
for (initialization;
     condition;
     increment)
{
   // block of code
}
```

4. foreach Loop

```
foreach (dataType item
         in collection)
{
   // block of code
}
```

3. Jump Statements

- break → Terminates the loop or switch.
- continue → Skips the current iteration and moves to the next iteration.
- return → Exits from the current method and returns control to caller.
- goto → Jumps to a labeled statement within the same method.

Example Program

```
using System;
class Program
{
   static void Main(string[] args)
   {
      int i;
      for (i = 1; i <= 5; i++)
      {
        if (i == 3) continue;   // skip 3
        Console.WriteLine(i);   // prints 1 2 4 5
      }
   }
}
```

Key Points :

- ✓ Selection statements help in decision making.
- ✓ Iteration statements help in repeating tasks.
- ✓ Jump statements transfer control instantly.
- ✓ Proper use of control statements makes code efficient and readable.

@codedsoul_05

4. Methods & Parameters

What are Methods?

A method is a block of code that performs a specific task. Methods help in code reusability, better organization and readability.

Note :
Every program is made up of one or more methods.
Main() is also a method.

Why Use Methods?

- Reusability of code
- Improves readability
- Easier debugging
- Modularity and maintainability

Basic Syntax

```
returnType MethodName(parameterList)
{
    // statements
    return value;   // if returnType is not void
}
```

→ returnType : Type of value returned by the method (void if nothing returned)
→ MethodName : Name of the method
→ parameterList : List of parameters, if any

Example Program

```
using System;
class Program
{
   // Main method - Entry point
   static void Main(string[] args)
   {
      int sum = Add(10, 20);    // Calling method
      Console.WriteLine("Sum = " + sum);
      Greet("Nikhil");
   }
   // Method with return type
   static int Add(int a, int b)
   {
      return a + b;
   }
   // Method with no return type
   static void Greet(string name)
   {
      Console.WriteLine("Hello, " + name + "!");
   }
}
```

Types of Methods

Type	Description	Example
1. Built-in Methods	Provided by C# / .NET library.	Console.WriteLine(), Math.Sqrt(), string.Length
2. User-defined Methods	Created by the developer to perform specific tasks.	Add(), Greet(), DisplayInfo()
3. Static Methods	Belong to the class rather than objects. Called using class name.	Math.Pow(), Program.Main()
4. Instance Methods	Belong to objects. Need object to call.	obj.Display(), obj.Calculate()

Parameters in Methods

1. No Parameters
Method does not take any input.

```
void Show()
{
   Console.WriteLine("Hi");
}
```

2. Parameterized
Method takes input through parameters.

```
void Print(string name)
{
   Console.WriteLine(name);
}
```

3. Multiple Parameters
Method takes more than one parameter.

```
int Multiply(int a, int b)
{
   return a * b;
}
```

4. Optional Parameters
Parameters with default values.

```
void Show(int x = 10)
{
   Console.WriteLine(x);
}
```

5. Named Arguments
Pass arguments by naming parameters.

```
Show(x: 20);
Show(x: 20, y: 30);
```

Key Points :

- ✓ Methods break big problems into small, manageable tasks.
- ✓ Use meaningful names for methods and parameters.
- ✓ Keep methods focused on one task.
- ✓ Use return type properly.
- ✓ Good methods make code clean, reusable and efficient.

Example with Parameters

```
using System;
class Program
{
   static int Subtract(int a, int b)          ← Method with two parameters
   {
      return a - b;
   }

   static void Main(string[] args)
   {
      int result = Subtract(50, 15);          ← Calling method and passing arguments
      Console.WriteLine("Result = " + result);
   }
}
```

@codedsoul_05

5. Arrays & Strings

Note :
Arrays store multiple values of the same type.
Strings store text.

What are Arrays?

An array is a collection of elements of the same data type, stored in contiguous memory locations. Elements are accessed using index.

Why Use Arrays?

- Store multiple values of same type
- Easy to access using index
- Useful for loops and processing collections
- Foundation for other data structures

Array Declaration & Initialization

`int[] numbers;`	`// Declaration`
`numbers = new int[5];`	`// Creating array of size 5`
`int[] nums = new int[5];`	`// Declaration + creation`
`int[] arr = { 1, 2, 3, 4, 5 };`	`// Declaration + initialization`

Array Indexing

- Index starts from 0
- Last index = length - 1
- Access: arrayName[index]
- Modify: arrayName[index] = value;

Example :

Index:	0	1	2	3	4
Value:	10	20	30	40	50

Example Program

```
using System;
class Program
{
    static void Main(string[] args)
    {
        int[] marks = { 85, 90, 78, 92, 88 };
        int sum = 0;
        for (int i = 0; i < marks.Length; i++)
        {
            Console.WriteLine($"Index {i} = {marks[i]}");
            sum += marks[i];
        }
        double avg = (double)sum / marks.Length;
        Console.WriteLine($"Average = {avg}");
    }
}
```

Common Array Properties & Methods

Property / Method	Description
Length	Returns the number of elements in array.
Sort()	Sorts the array in ascending order.
Reverse()	Reverses the order of elements.
IndexOf(value)	Returns the index of the first occurrence.
Clear(array, index, count)	Sets a range of elements to default.
Copy(source, dest, length)	Copies elements from one array to another.

What are Strings?

A string is a sequence of characters. In C#, string is an object of System.String class.

String Declaration

```
string name = "Nikhil";
string greeting = "Hello, World!";
string empty = string.Empty;
```

Common String Properties & Methods

Property / Method	Description
Length	Returns number of characters.
ToUpper()	Converts to uppercase.
ToLower()	Converts to lowercase.
Trim()	Removes leading and trailing spaces.
Substring(start, length)	Returns part of the string.
Contains(value)	Checks if string contains value.
Replace(old, new)	Replaces all occurrences.
Split(separator)	Splits string into array.

String Example Program

```
using System;
class Program
{
    static void Main(string[] args)
    {
        string text = " Hello C# World! ";
        string name = "Nikhil";

        Console.WriteLine(text.Trim());          // Hello C# World!
        Console.WriteLine(text.ToUpper());       //  HELLO C# WORLD!
        Console.WriteLine(text.ToLower());       //  hello c# world!
        Console.WriteLine(text.Contains("C#"));  // True
        Console.WriteLine(text.Replace("C#", "C sharp"));
        Console.WriteLine($"Length = {text.Trim().Length}");

        string[] words = text.Trim().Split(' ');
        Console.WriteLine($"Words Count = {words.Length}");
    }
}
```

Key Points

- ✓ Arrays hold multiple values of same type.
- ✓ Indexing is zero-based.
- ✓ Strings are immutable (cannot be changed directly).
- ✓ Use string methods to perform operations on text.
- ✓ Choose arrays for fixed-size collections and strings for text data.

@codedsoul_05

6. Classes & Objects

Note :
Everything in C# is built around classes and objects.

What is a Class?

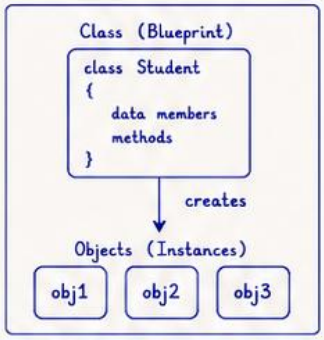
A class is a blueprint or template that defines data (variables) and behavior (methods). It is used to create objects.

What is an Object?

An object is an instance of a class. It has its own state (data) and can use the behavior defined in the class.

Why Use Classes & Objects?

- Encapsulation of data and methods
- Code reusability
- Better organization
- Real-world modeling
- Basis of OOP in C#



Class Declaration

```
public class Student
{
    // Data members (fields)
    public string name;
    public int age;

    // Method
    public void DisplayInfo()
    {
        Console.WriteLine($"Name: {name}, Age: {age}");
    }
}
```

Fields (data)

Method (behavior)

Creating Objects

```
Student s1 = new Student();      // Object of class Student
s1.name = "Nikhil";
s1.age = 20;
s1.DisplayInfo();                // Calling method
```

Key Points

- ✓ Class is a blueprint.
- ✓ Object is an instance of class.
- ✓ Use 'new' keyword to create objects.
- ✓ Access members using dot (.) operator.
- ✓ Each object has its own copy of data.

Example Program

```
using System;
class Program
{
    public class Student
    {
        public string name;
        public int age;
        public void DisplayInfo()
        {
            Console.WriteLine($"Name: {name}, Age: {age}");
        }
    }
    static void Main(string[] args)
    {
        Student s1 = new Student();
        s1.name = "Nikhil";
        s1.age = 20;
        s1.DisplayInfo();

        Student s2 = new Student();
        s2.name = "Aman";
        s2.age = 21;
        s2.DisplayInfo();
    }
}
```

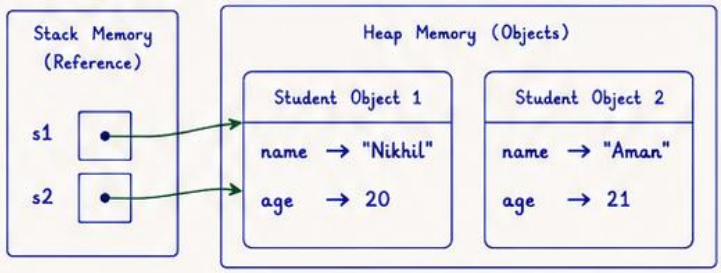
Output :

```
Name: Nikhil, Age: 20
Name: Aman, Age: 21
```

Accessing Members

Access	Description	Example
.	Used to access members of object.	obj.member
new Keyword	Creates a new object in memory.	ClassName obj = new ClassName();
this Keyword	Refers to current object instance.	this.member

Memory Representation



Key Takeaways

- ✓ Class defines structure and behavior.
- ✓ Object is a real-world entity created from class.
- ✓ Use objects to access fields and call methods.
- ✓ Foundation for all OOP concepts in C#.

@codedsoul_05

7. Constructors, Properties & Access Modifiers

Note :
Constructors initialize objects, Properties provide controlled access, and Access Modifiers control visibility.

• Constructors in C#

A constructor is a special method that is automatically called when an object is created. It initializes the object.

Types of Constructors

1. Default Constructor (no parameters)
2. Parameterized Constructor
3. Copy Constructor (user-defined)
4. Static Constructor (for class initialization)

• Why Use Constructors?

- Initialize object state
- Ensure objects start in a valid state
- Reduce code redundancy
- Improve code readability

• Example

```
public class Student
{
   public string Name;
   public int Age;

   // 1. Default Constructor
   public Student()                          // Sets default
   {                                         // values
      Name = "Unknown";
      Age = 0;
   }

   // 2. Parameterized Constructor
   public Student(string name, int age)      // Sets values using
   {                                         // parameters
      Name = name;
      Age = age;
   }

   // 3. Copy Constructor
   public Student(Student other)             // Creates new object
   {                                         // from existing object
      Name = other.Name;
      Age = other.Age;
   }
   // 4. Static Constructor
   static Student()                          // Called once when
   {                                         // class is first used
      Console.WriteLine("Student class loaded.");
   }
}
```

• Using Constructors

```
class Program
{
   static void Main(string[] args)
   {
      Student s1 = new Student();            // Default
      Student s2 = new Student("Nikhil", 20);  // Parameterized
      Student s3 = new Student(s2);          // Copy

      Console.WriteLine($"s1: {s1.Name}, {s1.Age}");
      Console.WriteLine($"s2: {s2.Name}, {s2.Age}");
      Console.WriteLine($"s3: {s3.Name}, {s3.Age}");
   }
```

Output :

```
Student class loaded.

s1: Unknown, 0
s2: Nikhil, 20
s3: Nikhil, 20
```

• Properties in C#

Properties provide a flexible way to read, write, or compute the value of a private field.

```
public class Student
{
   private string name;

   // Property with get and set accessors
   public string Name
   {
      get { return name; }          // Read value
      set { name = value; }         // Write value
   }

   // Read-only property
   public int Age { get; private set; }
}
```

Example

```
Student s = new Student();
s.Name = "Aman";          // using set
Console.WriteLine(s.Name); // using get
```

Types of Properties

1. Read-Write : get and set
2. Read-Only : only get
3. Write-Only : only set
4. Auto-Implemented: { get; set; }

• Access Modifiers in C#

Access modifiers define the visibility or accessibility of classes, fields, methods, and other members.

Modifier	Accessibility	Within Class	Within Same Assembly	Derived Class (Other Assembly)
public	Accessible everywhere	✓	✓	✓
private	Accessible only within class	✓	✗	✗
protected	Accessible within class and derived classes	✓	✓	✗
internal	Accessible within the same assembly	✓	✓	✗
protected internal	Accessible within same assembly or in derived classes	✓	✓	✗
private protected	Accessible within same assembly and in derived classes	✓	✓	✗

Example

```
public class Base
{
   protected int x = 10;
   private int y = 20;
   internal int z = 30;
   public int p = 40;
}
public class Derived : Base
{
   void Show()
   {
      Console.WriteLine(x);  // OK
      // Console.WriteLine(y);  // Error
      Console.WriteLine(z);  // OK (same assembly)
      Console.WriteLine(p);  // OK
   }
}
```

Key Takeaways

- ✓ Constructors initialize objects and are called automatically.
- ✓ Properties provide controlled access to fields.
- ✓ Access modifiers ensure data hiding and security.
- ✓ Use the right modifier to follow best practices of OOP.

@codedsoul_05

8. Inheritance & Polymorphism

Note :
Inheritance promotes code reusability. Polymorphism provides flexibility.

What is Inheritance?

Inheritance is a mechanism where a new class (derived class) acquires the properties and behaviors (fields and methods) of an existing class (base class).

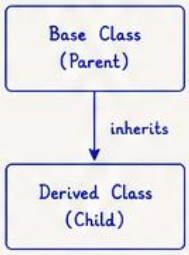
Why Use Inheritance?

- Code reusability
- Extensibility
- Method overriding (run-time polymorphism)
- Represents IS-A relationship

Types of Inheritance in C#

1. Single Inheritance : One base → one derived
2. Multilevel Inheritance : A → B → C
3. Hierarchical Inheritance : One base → multiple derived
4. Multiple Inheritance : Not supported in C# (can be achieved using Interfaces)
5. Hybrid Inheritance : Combination of above types (via interfaces)

Inheritance Relationship



Inheritance Example (Single Inheritance)

```
using System;

// Base class
class Animal
{
    public void Eat()
    {
        Console.WriteLine("Animal is eating");
    }
}

// Derived class
class Dog : Animal      // Dog IS-A Animal
{
    public void Bark()
    {
        Console.WriteLine("Dog is barking");
    }
}

class Program
{
    static void Main()
    {
        Dog d = new Dog();
        d.Eat();    // Inherited method
        d.Bark();   // Own method
    }
}
```

Output :

```
Animal is eating
Dog is barking
```

Key Points

- ✓ Derived class inherits members of base class.
- ✓ Private members of base class are not accessible.
- ✓ Use ':' to inherit.
- ✓ Supports method overriding.

Polymorphism in C#

Polymorphism means "many forms". It allows objects to be treated as instances of their parent class. There are two types:

1. Compile-time Polymorphism (Method Overloading)
2. Run-time Polymorphism (Method Overriding)

1. Compile-time Polymorphism (Overloading)

- Same method name, different parameters.

```
class Calculator
{
    public int Add(int a, int b)
    { return a + b; }                       // two integers

    public double Add(double a, double b)
    { return a + b; }                       // two doubles

    public int Add(int a, int b, int c)
    { return a + b + c; }                   // three integers
}

// Compiler decides which method to call.
```

Run-time Polymorphism (Overriding)

- Achieved using inheritance and virtual methods.
- The method to execute is decided at run-time.

```
class Animal
{
    public virtual void Sound()      // virtual method
    {
        Console.WriteLine("Some sound");
    }
}
class Dog : Animal
{
    public override void Sound()     // overriding
    {
        Console.WriteLine("Bark");
    }
}
class Program
{
    static void Main()
    {
        Animal a = new Dog();   // base class reference
        a.Sound();              // calls Dog.Sound()
    }
}
```

Output :

```
Bark
```

Rules for Method Overriding

- The base method must be marked as virtual.
- The derived method must use override keyword.
- Same method signature (name, parameters, return type).
- Access level in derived class cannot be more restrictive.

Abstract Class & Method

- Use abstract to declare a class or method without implementation.
- Derived class must provide implementation.

```
abstract class Shape
{
    public abstract double Area();      // no body
}

class Circle : Shape
{
    public override double Area()
    {
        return 3.14 * 5 * 5;
    }
}
```

Types of Polymorphism

Type	Binding Time	Achieved By	Example
Compile-time	At compile time	Method Overloading, Operator Overloading	Add(int, int) Add(int, int, int)
Run-time	At run time	Method Overriding (virtual + override)	Animal a = new Dog(); a.Sound();

Key Takeaways

- ✓ Inheritance = Reuse + Extensibility.
- ✓ Polymorphism = Flexibility.
- ✓ Use virtual & override for run-time polymorphism.
- ✓ Interfaces can be used to achieve multiple inheritance.

@codedsoul_05

9. Abstraction & Interfaces

Note :
Abstraction hides implementation details and shows only the essential features.

What is Abstraction?

Abstraction is the process of hiding the internal implementation and showing only the functionality to the user.

It can be achieved in C# using:

1. Abstract Classes
2. Interfaces

Why Use Abstraction?

- Hides complexity
- Focus on what an object does, not how it does
- Improves security
- Easier maintenance
- Promotes loose coupling

Abstraction vs Encapsulation

Feature	Abstraction	Encapsulation
Purpose	Hides implementation details	Hides data and implementation details
Focus	On behavior	On data + behavior
Achieved By	Abstract class & Interface	Access modifiers, properties
When Used	When you want to define a contract	When you want to restrict data access

Abstract Classes

- An abstract class cannot be instantiated.
- It can have abstract (no body) and concrete methods.
- Derived class must implement all abstract methods.

```
using System;
// Abstract class
abstract class Shape
{
    // Abstract method (no body)
    public abstract double Area();          // must be implemented
                                            // by derived class
    // Concrete method
    public void Display()
    {
        Console.WriteLine("This is a shape.");
    }
}

// Derived class
class Circle : Shape
{
    public override double Area()
    {
        return 3.14 * 5 * 5;
    }
}
class Program
{
    static void Main()
    {
        // Shape s = new Shape();      // Error: cannot create
        Circle c = new Circle();       // object of abstract class
        Console.WriteLine("Area = " + c.Area());
        c.Display();
    }
}
```

Output :
Area = 78.5
This is a shape.

Key Points (Abstract Class)

- ✓ Cannot create an object of abstract class.
- ✓ Can have fields, properties, methods (abstract + concrete).
- ✓ Derived class must use 'override' for abstract methods.
- ✓ Supports constructors.

Interfaces

- An interface is a contract. It contains only method signatures.
- By default, members are public and abstract.
- A class can implement multiple interfaces.

```
// Interface declaration
public interface IShape
{
    double Area();
    void Display();
}
// Class implementing interface
class Rectangle : IShape
{
    public double Area()
    {
        return 10 * 5;
    }
    public void Display()
    {
        Console.WriteLine("Rectangle Shape");
    }
}

class Program
{
    static void Main()
    {
        IShape s = new Rectangle();  // interface reference
        Console.WriteLine("Area = " + s.Area());
        s.Display();
    }
}
```

Output :
Area = 50
Rectangle Shape

Key Points (Interface)

- ✓ Cannot create an object of interface.
- ✓ Contains only method signatures (no body).
- ✓ Members are implicitly public and abstract.
- ✓ A class can implement multiple interfaces.
- ✓ Supports properties, indexers, events (C# 8.0+ default methods).

Example – Abstract Class vs Normal Class

Abstract Class

```
abstract class Animal
{
    public abstract void Sound();
    public void Sleep()
    {
        Console.WriteLine("Animal is sleeping");
    }
}
```

Normal Class

```
class Animal
{
    public void Sound()
    {
        Console.WriteLine("Some sound");
    }
}
```

Abstract class forces derived class to provide specific implementation, while normal class provides default implementation.

Multiple Interfaces

```
public interface IPrintable
{
    void Print();
}
public interface IScannable
{
    void Scan();
}
class MultiFunctionPrinter : IPrintable, IScannable
{
    public void Print() => Console.WriteLine("Printing...");
    public void Scan()  => Console.WriteLine("Scanning...");
}
```

Output :
Printing...
Scanning...

Abstract Class vs Interface

Feature	Abstract Class	Interface
Instantiation	Cannot be instantiated	Cannot be instantiated
Members	Fields, properties, methods, constructors, destructors	Methods, properties, indexers, events (no fields)
Access Modifiers	Can have any access level	Public by default
Inheritance	Single inheritance only	Multiple implementation
Use When	You want to provide partial implementation	You want to define a contract/capability

Key Takeaways

- ✓ Abstraction focuses on behavior, not implementation.
- ✓ Use abstract classes when you want to share common code.
- ✓ Use interfaces when you want to define a capability or contract.
- ✓ Interfaces make the system more flexible and loosely coupled.

@codedsoul_05

10. Encapsulation & Advanced OOP

Note :
Encapsulation protects data using access modifiers. Advanced OOP concepts make code more powerful and scalable.

What is Encapsulation?

Encapsulation is the process of wrapping data (fields) and methods together as a single unit and restricting direct access to some of an object's components.

Why Use Encapsulation?

- Protects data from unintended changes
- Improves code maintainability
- Achieves better control over object state
- Supports data hiding

Access Modifiers Recap

Modifier	Same Class	Same Assembly	Derived Class (Other Assembly)	Outside Assembly
public	✓	✓	✓	✓
private	✓	✗	✗	✗
protected	✓	✓	✓	✗
internal	✓	✓	✗	✗
protected internal	✓	✓	✓ (via derivation)	✓ (via reference)
private protected	✓	✗	✓ (via derivation)	✗

Encapsulation Example

```
using System;
class BankAccount
{
    private decimal balance;                    // private field

    // Public property to read balance
    public decimal Balance
    {
        get { return balance; }                 // read-only
    }

    // Public method to deposit
    public void Deposit(decimal amount)
    {
        if (amount > 0)
            balance += amount;
    }

    // Public method to withdraw
    public bool Withdraw(decimal amount)
    {
        if (amount > 0 && amount <= balance)
        {
            balance -= amount;
            return true;
        }
        return false;
    }
}
class Program
{
    static void Main()
    {
        BankAccount acc = new BankAccount();
        acc.Deposit(1000);
        acc.Withdraw(200);
        Console.WriteLine($"Balance: {acc.Balance}");
    }   // acc.balance = 5000;    // Error! private field
}
```

Output :
Balance: 800

Advanced OOP Concepts

- Static Members
- Readonly & Const
- Sealed Class & Methods
- Abstract Class vs Interface (recap)
- Nested (Inner) Classes
- Partial Classes
- Extension Methods
- Operator Overloading
- Generics
- Delegates & Events (basics)

Static Members

```
class Counter
{
    public static int Count = 0;    // shared by all objects

    public Counter()
    {
        Count++;
    }
}

class Program
{
    static void Main()
    {
        new Counter();
        new Counter();
        Console.WriteLine($"Count: {Counter.Count}");
    }
}
```

Output :
Count: 2

Readonly vs Const

Feature	Readonly	Const
Value set	At runtime (constructor)	At compile time
Can be changed	No	No
Where used	Fields	Fields, local vars, params
Must initialize	Yes (in declaration or constructor)	Yes (at declaration)

Nested (Inner) Class

```
class Outer
{
    public int x = 10;

    public class Inner
    {
        public void Display(Outer o)
        {
            Console.WriteLine(o.x);
        }
    }
}
```

Partial Class

```
// File1.cs
partial class Employee
{
    public string Name;
}

// File2.cs
partial class Employee
{
    public void Display()
    {
        Console.WriteLine(Name);
    }
}
```

Extension Method

```
static class StringExtensions
{
    public static int WordCount(
      this string s)
    {
        return s.Split(' ',
            StringSplitOptions.
            RemoveEmptyEntries)
                .Length;
    }
}

// Usage
string text = "hello world";
Console.WriteLine(text.
WordCount());   // 2
```

Sealed Class & Methods

Sealed Class – Cannot be inherited.

```
sealed class Logger
{
    public void Log()
    {
        Console.WriteLine("Logging...");
    }
}

// Error! Cannot inherit
class FileLogger : Logger { }
```

Sealed Method – Cannot be overridden.

```
class Base
{
    public virtual void Show() { }
    public sealed void Print() { }
}

class Derived : Base
{
    public override void Show() { }   // OK
    // public override void Print() { } // Error!
}
```

Generics

```
class Box<T>
{
    private T item;
    public void Set(T i) { item = i; }
    public T Get() { return item; }
}

// Usage
Box<int> intBox = new Box<int>();
intBox.Set(10);
Console.WriteLine(intBox.Get());   // 10

Box<string> strBox = new Box<string>();
strBox.Set("Hello");
Console.WriteLine(strBox.Get());   // Hello
```

Delegates & Events (Basics)

```
// Delegate
public delegate void Notify(string msg);

class MessageService
{
    public event Notify OnNotify;

    public void Send(string msg)
    {
        OnNotify?.Invoke(msg);  // raise event
    }
}
class Program
{
    static void Main()
    {
        MessageService svc = new MessageService();
        svc.OnNotify += (m) => Console.WriteLine($"Notified: {m}");
        svc.Send("Hello Coders!");
    }
}
```

Operator Overloading

```
class Complex
{
    public int Real, Imag;

    public Complex(int r, int i)
    {  Real = r; Imag = i; }

    public static Complex operator +(Complex a, Complex b)
    {
        return new Complex(a.Real + b.Real, a.Imag + b.Imag);
    }
}
// Usage
Complex c1 = new Complex(2, 3);
Complex c2 = new Complex(4, 5);
Complex c3 = c1 + c2;   // (6, 8)
```

Key Takeaways

- ✓ Encapsulation hides data and exposes only what is necessary.
- ✓ Use access modifiers to control visibility.
- ✓ Advanced OOP features make code more reusable and maintainable.
- ✓ Understand when to use classes, interfaces, and abstract classes.
- ✓ Practice concepts like generics, delegates, and events and operator overloading.
- ✓ Write clean, secure, and scalable object-oriented code.

@codedsoul_05

11. Collections & Generics

Note :
Collections store and manage groups of objects efficiently. Generics enable type safety and reusability.

What is a Collection?

A collection is an object that stores multiple objects of the same type and provides methods to add, remove, search, and manipulate them.

Why Use Collections?

- ✓ Store multiple items
- ✓ Built-in methods for common operations
- ✓ Better performance and scalability
- ✓ Type safety and code reusability (with Generics)

Types of Collections in .NET

Category	Interface / Base	Description	Examples
List	IList<T>, List<T>	Ordered collection (index based)	List<T>
Set	ISet<T>, HashSet<T>	No duplicates	HashSet<T>
Dictionary	IDictionary<TKey,TValue>, Dictionary<TKey,TValue>	Key-Value pairs (unique keys)	Dictionary<TKey, TValue>
Queue	Queue<T>	First In, First Out (FIFO)	Queue<T>
Stack	Stack<T>	Last In, First Out (LIFO)	Stack<T>

Common Operations

Operation	Meaning
Add()	Adds an item
Remove()	Removes an item
Contains()	Checks existence
Clear()	Removes all items
Count / Length	Gets number of items
[] (Indexer)	Access item by index / key

List<T> Example

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
        List<string> names = new List<string>();   // Create list
        names.Add("Nikhil");
        names.Add("Aman");
        names.Add("Rahul");

        Console.WriteLine("Count: " + names.Count);     // 3
        Console.WriteLine("First: " + names[0]);        // Nikhil
        Console.WriteLine("Contains Aman? " + names.Contains("Aman")); // True

        names.Remove("Rahul");
        foreach(string name in names)
            Console.WriteLine(name);

    }
}
```

Output :
```
Count: 3
First: Nikhil
Contains Aman? True
Nikhil
Aman
```

Dictionary<TKey, TValue> Example

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
        Dictionary<int, string> students = new Dictionary<int, string>();
        students.Add(1, "Nikhil");
        students.Add(2, "Aman");
        students.Add(3, "Rahul");

        Console.WriteLine(students[2]);            // Aman
        Console.WriteLine("Contains key 3? " + students.ContainsKey(3)); // True

        students[2] = "Aman Sharma";       // Update value

        foreach(var kvp in students)
            Console.WriteLine(kvp.Key + " -> " + kvp.Value);
    }
}
```

Output :
```
Aman
Contains key 3? True
1 -> Nikhil
2 -> Aman Sharma
3 -> Rahul
```

Other Collection Examples

HashSet<T> (No Duplicates)
```
HashSet<int> set =
    new HashSet<int>();

set.Add(10);
set.Add(20);
set.Add(10);   // duplicate

Console.WriteLine(set.Count);
   // Output: 2
```

Queue<T> (FIFO)
```
Queue<string> q =
    new Queue<string>();

q.Enqueue("A");
q.Enqueue("B");
q.Enqueue("C");

Console.WriteLine(q.Dequeue());
 // Output: A
```

Stack<T> (LIFO)
```
Stack<int> st =
    new Stack<int>();

st.Push(10);
st.Push(20);
st.Push(30);

Console.WriteLine(st.Pop());
 // Output: 30
```

LinkedList<T>
```
LinkedList<int> list =
    new LinkedList<int>();

list.AddLast(1);
list.AddLast(2);
list.AddFirst(0);

// 0 <-> 1 <-> 2
```

Generics in C#

- Generics allow you to write classes, methods, and interfaces with placeholders for types.
- Provides type safety and reusability.
- Eliminates boxing/unboxing for value types.

Generic Class Example

```
class Box<T>
{
    private T item;              // field of type T
    public void Set(T value)     // method using T
        { item = value; }
    public T Get()               // method returning T
        { return item; }
}

class Program
{
    static void Main()
    {
        Box<int> intBox = new Box<int>();
        intBox.Set(100);
        Console.WriteLine(intBox.Get());    // 100

        Box<string> strBox = new Box<string>();
        strBox.Set("Hello");
        Console.WriteLine(strBox.Get());    // Hello
    }
}
```

Output :
```
100
Hello
```

Generic Method Example

```
class Utils
{
    public static T GetFirst<T>(List<T> list)
    {
        if (list == null || list.Count == 0)
            throw new ArgumentException("List is empty");
        return list[0];
    }
}

class Program
{
    static void Main()
    {
        List<int> nums = new List<int> { 10, 20, 30 };
        List<string> names = new List<string> {
          "Aman", "Nikhil" };

        Console.WriteLine(Utils.GetFirst(nums));    // 10
        Console.WriteLine(Utils.GetFirst(names));   // Aman
    }
}
```

Output :
```
10
Aman
```

Generic Interface Example

```
interface IRepository<T>
{
    void Add(T item);
    T Get(int id);
}

class Repository<T> : IRepository<T>
{
    private List<T> items = new List<T>();

    public void Add(T item) => items.Add(item);
    public T Get(int id) => items[id];
}

class Program
{
    static void Main()
    { IRepository<string> repo = new Repository<string>();
      repo.Add("Nikhil");
      Console.WriteLine(repo.Get(0));    // Nikhil
    }
}
```

Output :
```
Nikhil
```

Key Takeaways

- ✓ Collections help store and manage multiple objects efficiently.
- ✓ Choose the right collection based on requirements.
- ✓ Generics provide type safety, reusability, and better performance.
- ✓ Use collections + generics to write clean, scalable, and maintainable code.

When to Use Which Collection?

Collection	Use When
List<T>	You need a dynamic array with fast access by index and order is important.
HashSet<T>	You need unique items and fast lookup (no duplicates).
Dictionary<TKey,TValue>	You need to store key-value pairs and quickly look up by key.
Queue<T>	You need to process items in FIFO order.
Stack<T>	You need to process items in LIFO order.
LinkedList<T>	You need frequent insertions/deletions in the middle.

Collection vs Array

Feature	Array	Collection (e.g., List<T>)
Size	Fixed	Dynamic
Type	Stores elements of same type	Can store objects (same type) with more features
Methods	Limited	Rich set of built-in methods
Ease of Use	Less flexible	More flexible and convenient
Performance	Faster for fixed size	Optimized for dynamic scenarios

@codedsoul_05

12. Delegates, Events & LINQ

Note :
Delegates reference methods. Events notify subscribers. LINQ lets you query data in a simple, powerful way.

• Delegates

A delegate is a type-safe function pointer. It can reference methods with a specific signature.

```
// 1. Declare delegate
public delegate int Operation(int a, int b);

// 2. Methods matching the signature
public static int Add(int x, int y) => x + y;
public static int Multiply(int x, int y) => x * y;

// 3. Use delegate
Operation op = Add;          // reference method
Console.WriteLine(op(5, 3));     // 8
op = Multiply;
Console.WriteLine(op(5, 3));     // 15
```

Built-in Generic Delegates	
Delegate	Description
Action	Represents a method with no return value
Action<T1, T2, ...>	Method with parameters, no return value
Func<TResult>	Method with no params, returns TResult
Func<T1, TResult>	Method with 1 param, returns TResult
Func<T1, T2, TResult>	Method with 2 params, returns TResult
Predicate<T>	Method that returns bool for type T

Key Points - Delegates

- ✓ Type-safe
- ✓ Can be assigned, passed as parameters and returned from methods
- ✓ Support multicast (multiple methods)
- ✓ Provide flexibility and loose coupling

• Multicast Delegates

Delegates can reference multiple methods. All methods are invoked in order.

```
public static void First() => Console.WriteLine("First");
public static void Second() => Console.WriteLine("Second");

Action act = First;
act += Second;          // add another method
act();

// Output:
// First
// Second
```

• Events

An event is a special kind of delegate that restricts direct invocation from outside the class.

```
public class Button
{
    public event EventHandler Clicked;

    public void OnClick()
    {
        Clicked?.Invoke(this, EventArgs.Empty);
    }
}

// Subscriber
button.Clicked += (s, e) =>
   Console.WriteLine("Button clicked!");

// Trigger
button.OnClick();    // Output: Button clicked!
```

Key Points - Events

- ✓ Enable publish-subscribe pattern
- ✓ Only declaring class can raise the event
- ✓ Prevents external code from invoking the event directly

• Events Example (Complete)

```
using System;
public class StockNotifier
{
    public event Action<string> PriceChanged;

    private decimal price;

    public decimal Price
    {
        get => price;
        set
        {
            if (price != value)
            {
                price = value;
                PriceChanged?.Invoke($"Price updated: {price}");
            }
        }
    }
}
class Program
{
    static void Main()
    {
        StockNotifier stock = new StockNotifier();
        stock.PriceChanged += msg => Console.WriteLine(msg);
        stock.Price = 100;    // Price updated: 100
        stock.Price = 120;    // Price updated: 120
    }
}
```

Output :
Price updated: 100
Price updated: 120

• LINQ (Language Integrated Query)

LINQ allows querying collections, arrays, databases, XML and more using a simple, declarative syntax.

1. Query Syntax (SQL-like)

```
var numbers = new List<int> { 1, 2, 3, 4, 5, 6, 7, 8 };

var result = from n in numbers
             where n % 2 == 0
             select n;

foreach (var n in result)
   Console.Write(n + " ");    // 2 4 6 8
```

2. Method Syntax (Fluent)

```
var result = numbers
    .Where(n => n % 2 == 0)
    .Select(n => n);

foreach (var n in result)
   Console.Write(n + " ");    // 2 4 6 8
```

Query Operators (Common)

Operator	Purpose
Where()	Filters elements based on condition
Select()	Projects each element to new form
SelectMany()	Projects and flattens sequences
OrderBy()	Sorts in ascending order
OrderByDescending()	Sorts in descending order
GroupBy()	Groups elements by a key
Join()	Performs inner join
Any()	Checks if any element matches
All()	Checks if all elements match
Count()	Counts the number of elements
First()/FirstOrDefault()	Gets first element
Last()/LastOrDefault()	Gets last element
Distinct()	Returns distinct elements
ToList()/ToArray()	Converts result to List/Array

• LINQ Examples

1. Filtering

```
var nums = new List<int>
  { 10, 15, 20, 25, 30 };

var res = nums.Where(n => n > 15);
}

// Output: 20 25 30
```

2. Projection

```
var names = new List<string>
    { "Aman", "Nikhil", "Rahul" };

var res = names.Select(n => n.ToUpper());
}

// Output: AMAN NIKHIL RAHUL
```

3. Ordering

```
var nums = new List<int>
  { 5, 2, 9, 1, 7 };

var res = nums.OrderBy(n =>
n);

// Output: 1 2 5 7 9
```

4. Grouping

```
var items = new[] {
   new { Dept = "HR", Name = "Aman" },
   new { Dept = "IT", Name = "Nikhil" },
   new { Dept = "HR", Name = "Rahul" }
};

var res = items.GroupBy(i => i.Dept);

// Output:
// HR -> Aman, Rahul
// IT -> Nikhil
```

5. Joining

```
var emp = new[] { new { Id = 1, Name = "Aman" },
          new[] { Id = 2, Name = "Nikhil" } };

var dept = new[] { new { Id = 1, Dept = "HR" },
           new[] { Id = 2, Dept = "IT" } };

var res = from e in emp
          join d in dept on e.Id equals d.Id
          select new { e.Name, d.Dept };

// Output:
// Aman - HR
// Nikhil - IT
```

6. Aggregation

```
var nums = new List<int>
  { 1, 2, 3, 4, 5 };

int sum = nums.Sum();
// 15
double avg = nums.Average();
// 3
int count = nums.Count();
// 5
```

• Delegate vs Event

Feature	Delegate	Event
Purpose	Reference methods	Notify subscribers
Invocation	Can be invoked from anywhere	Only declaring class can invoke
Access	Public by default	Controlled with add/remove
Pattern	Notifies by call	Follows observer pattern
Use When	Pass behavior as parameter	Notify multiple listeners about a change

• When to Use What?

Scenario	Use
Pass a method as a parameter	Delegate
Asynchronous callbacks	Delegate / Event
Notify when something happens	Event
Query / filter / transform data	LINQ
Complex data operations	LINQ

Key Takeaways

- ✓ Delegates are type-safe function pointers.
- ✓ Events enable the publish-subscribe model.
- ✓ LINQ makes querying collections and data easy and expressive.
- ✓ Choose the right tool for clean, maintainable and scalable code.

@codedsoul_05

13. Exception Handling & File Handling

Note :
Handle exceptions gracefully and always close/dispose resources (files, streams) properly.

Exception Handling Basics

Exception is a runtime error that disrupts the normal flow of the program.

```
try
{
    // code that may throw exception
}
catch (Exception ex)
{
    // handle exception
    Console.WriteLine("Error: " + ex.Message);
}
finally
{
    // always executed (cleanup code)
}
```

Key Points

- ✓ Improves program reliability
- ✓ Prevents abnormal termination
- ✓ Separate error-handling code from normal logic
- ✓ finally block is optional but useful for cleanup.

Common Exceptions in .NET

Exception	Description
Exception	Base class for all exceptions
ArithmeticException	Arithmetic errors (e.g., divide by zero)
NullReferenceException	Accessing a null object
IndexOutOfRangeException	Index out of range in array/collection
FormatException	Invalid format of data
InvalidCastException	Invalid type casting
FileNotFoundException	File not found on disk
IOException	I/O operation errors
ArgumentException	Invalid argument passed to method

Multiple Catch & Finally Example

```
try
{
    int[] arr = { 10, 20, 30 };
    Console.Write("Enter index: ");
    int i = int.Parse(Console.ReadLine());
    Console.WriteLine("Value = " + arr[i]);
}
catch (FormatException)
{
    Console.WriteLine("Invalid number format!");
}
catch (IndexOutOfRangeException)
{
    Console.WriteLine("Index is out of range!");
}
catch (Exception ex)
{
    Console.WriteLine("Something went wrong: " + ex.Message);
}
finally
{
    Console.WriteLine("This will always execute.");
}
```

Output :
Enter index: 5
Index is out of range!
This will always execute.

Custom Exception (User Defined)

```
using System;

public class InvalidAgeException : Exception
{
    public InvalidAgeException(string message) : base(message) { }
}

class Program
{
    static void ValidateAge(int age)
    {
        if (age < 18)
            throw new InvalidAgeException("Age must be 18 or above.");
    }

    static void Main()
    {
        try
        {
            ValidateAge(16);
        }
        catch (InvalidAgeException ex)
        {
            Console.WriteLine(ex.Message);
        }
    }
}
```

Output :
Age must be 18 or above.

Exception Handling Best Practices

- — Catch specific exceptions first, then general Exception.
- — Don't ignore exceptions.
- — Don't use exceptions for normal flow control.
- — Always log exceptions (at least during development).
- — Use finally (or using) to release resources.

File Handling Overview

The System.IO namespace provides classes to read, write, create, delete, and manipulate files.

Important Classes	Purpose
File	Static methods for file operations
FileInfo	Provide instance methods for file
Directory	Work with directories (folders)
Path	Utility methods for file paths
StreamReader / StreamWriter	Efficient reading/writing of text files
BinaryReader / BinaryWriter	Reading/writing binary data

File Handling - Text Files

1. Write Text to a File

```
using System.IO;

string path = "hello.txt";
File.WriteAllText(path, "Hello World!");
// Overwrites if file exists
// Creates new file if not exists
```

2. Read Text from a File

```
string text = File.ReadAllText(path);
Console.WriteLine(text);
```

3. Append Text to a File

```
File.AppendAllText(path,
        "\nThis line is appended.");
```

4. Read All Lines

```
string[] lines = File.ReadAllLines(path);
foreach (string line in lines)
        Console.WriteLine(line);
```

File Handling - Using StreamReader / StreamWriter

1. Read Using StreamReader

```
using System.IO;

using (StreamReader sr = new StreamReader("data.txt"))
{
    string line;
    while ((line = sr.ReadLine()) != null)
    {
        Console.WriteLine(line);
    }
}
```

2. Write Using StreamWriter

```
string System.IO;

using (StreamWriter sw = new StreamWriter("data.txt"))
{
    sw.WriteLine("Hello");
    sw.WriteLine("C# is Awesome!");
}
```

☆ Using 'using' statement ensures the file is closed automatically.

File Handling - Binary Files

1. Write Binary Data

```
using System.IO;

using (BinaryWriter bw = new BinaryWriter(
    File.Open("data.bin", FileMode.Create)))
{
    bw.Write(100);          // int
    bw.Write(3.14);         // double
    bw.Write("Nikhil");     // string
}
```

2. Read Binary Data

```
using System.IO;

using (BinaryReader br = new BinaryReader(
    File.Open("data.bin", FileMode.Open)))
{
    int id = br.ReadInt32();
    double price = br.ReadDouble();
    string name = br.ReadString();
    Console.WriteLine($"{id} {price} {name}");
}
```

Directory Operations

```
using System.IO;

// Create directory
Directory.CreateDirectory("MyFolder");

// Check if exists
bool exists = Directory.Exists("MyFolder");

// Get all files
String[] files = Directory.GetFiles("MyFolder");

// Delete directory (must be empty)
Directory.Delete("MyFolder", false);
```

Path Class Examples

```
string file = @"C:\Data\note.txt";
Path.GetFileName(file);                    // note.txt
Path.GetFileNameWithoutExtension(file);    // note
Path.GetExtension(file);                   // .txt
Path.GetDirectoryName(file);               // C:\Data
Path.Combine("C:\Data", "note.txt");
                        // C:\Data\note.txt
Path.GetFullPath("note.txt");
                        // full path
```

Exception vs Return Code

Aspect	Exception Handling	Return Code Handling
Readability	Clean and structured	Can be messy with many checks
Performance	Slightly slower	Faster
Best Use	For exceptional situations	For expected/normal flow
Maintenance	Easier to maintain	Harder with complex logic

Dos & Don'ts

Do ✅	Don't ❌
Handle specific exceptions	Catch Exception and ignore
Log exceptions	Throw exceptions without message
Dispose resources properly	Forget to close files/streams
Use using statement	Use exceptions for flow control

Key Takeaways

- ✓ Handle exceptions to make programs robust.
- ✓ Always release resources (files, streams).
- ✓ Use the right class for file operations.
- ✓ LINQ & IO together make data processing easy.
- ✓ Clean code = fewer exceptions & better handling! ☆

@codedsoul_05

14. Async, Await & Modern C#

Note :
Async makes I/O non-blocking.
Await makes async code look clean and sequential.
Modern C# = Productivity + Performance.

Why Async & Await?

- Avoids blocking threads during I/O operations
- Improves scalability and responsiveness
- Better performance in web, UI, and cloud apps
- Makes code look synchronous and easy to read

Key Concepts

Concept	Description
async	Marks a method as asynchronous
await	Pauses execution until the task completes
Task / Task<T>	Represents an async operation (with/without result)
async void	Used for event handlers (avoid in general methods)
ConfigureAwait(false)	Avoids capturing context (use in library code)
CancellationToken	Used to cancel async operations

Async vs Sync

Feature	Synchronous	Asynchronous
Thread	Blocks the thread	Does not block the thread
Performance	Lower (waits for I/O)	Higher (non-blocking)
Scalability	Poor	Excellent
Responsiveness	Low	High
Best Use	CPU-bound work	I/O-bound work

Basic Example

```
using System;
using System.Threading.Tasks;

class Program
{
    static async Task Main()
    {
        Console.WriteLine("Fetching data...");
        string data = await GetDataAsync();
        Console.WriteLine($"Data: {data}");
    }

    static async Task<string> GetDataAsync()
    {
        await Task.Delay(2000); // Simulate I/O (e.g., API call)
        return "Hello, Async World!";
    }
}
```

Output :
Fetching data...
(waits 2 sec)
Data: Hello, Async World!

Await in Action

```
static async Task DemoAsync()
{
    Console.WriteLine("Start");
    await Task.Delay(1000);
    Console.WriteLine("After 1s");

    await Task.Delay(1000); ⟷ await pauses the method here without blocking the thread. Other work can continue.
    Console.WriteLine("After 2s");
}

// Execution order:
// Start -> After 1s -> After 2s
// (No thread blocking during delays)
```

Returning Values

```
static async Task<int> GetNumberAsync()
{
    await Task.Delay(1000);
    return 42;
}

static async Task Demo()
{
    int num = await GetNumberAsync();
    Console.WriteLine(num); // 42
}
```

Running Multiple Tasks

1. Sequential (one by one)

```
var a = await Task1Async();
var b = await Task2Async();
var c = await Task3Async();      // slower
```

2. Parallel (concurrent)

```
var t1 = Task1Async();
var t2 = Task2Async();
var t3 = Task3Async();

await Task.WhenAll(t1, t2, t3);     // faster
var results = await Task.WhenAll(t1, t2, t3);
```

Use WhenAll for independent I/O operations.

Exception Handling in Async Code

```
static async Task DemoAsync()
{
    try
    {
        await Task.Delay(500);
        throw new Exception("Something went wrong!");
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Error: {ex.Message}");
    }
}
```

Exceptions in async methods propagate when awaited. Always use try-catch. ⚠

Cancellation with CancellationToken

```
static async Task DoWorkAsync(CancellationToken token)
{
    for (int i = 0; i < 5; i++)
    {
        token.ThrowIfCancellationRequested();
        Console.WriteLine($"Working... {i}");
        await Task.Delay(500, token);
    }
}
// Usage
var cts = new CancellationTokenSource();
var task = DoWorkAsync(cts.Token);
cts.CancelAfter(1200);  // cancel after 1.2 sec
try { await task; }
catch (OperationCanceledException)
{
    Console.WriteLine("Operation cancelled.");
}
```

Modern C# Features (Quick Highlights)

Feature	Introduced	What's New?	Example
Record Types	C# 9	Immutable reference types with value-based equality	public record Person(string Name, int Age);
Top-level Statements	C# 9	No need for Program class or Main method	Console.WriteLine("Hi from top-level statement!");
Init-only Properties	C# 9	Set value only during initialization	public string Name { get; init; }
Pattern Matching Enhancements	C# 9-12	More patterns: relational, logical, property patterns	if (obj is Person { Age: > 18 }) { }
File-scoped Namespaces	C# 10	Less indentation, cleaner code	namespace MyApp;
Global Using Directives	C# 10	Use common namespaces across files	global using System;
Nullable Reference Types	C# 8+	Helps avoid null reference exceptions	string? name = null;
Async Streams (IAsyncEnumerable<T>)	C# 8	Stream data asynchronously	await foreach (var item in GetDataAsync()) { }

Async Streams (IAsyncEnumerable<T>)

```
static async IAsyncEnumerable<int> GetNumbersAsync()
{
  for (int i = 1; i < 5; i++)
  {
      await Task.Delay(500);
      yield return i;
  }
}

// Consumption
await foreach (var num in GetNumbersAsync())
{
    Console.WriteLine(num);
}
```

Great for large data streams (e.g., reading files, network streams).

Task vs ValueTask

Aspect	Task	ValueTask
Type	Class	Struct
Allocation	Always allocates	May avoid allocation
Use When	General-purpose	High-performance scenarios
Caution	Safe to use multiple times	Should be awaited only once

async void vs async Task

Aspect	async Task	async void
Return Type	Task	void
Use	General methods	Event handlers only
Exception Handling	Can be caught	Cannot be caught
Awaitable	Yes	No

Best Practices

- ✓ Use async all the way (don't mix sync-over-async).
- ✓ Avoid async void (except for event handlers).
- ✓ Use ConfigureAwait(false) in library code.
- ✓ Use CancellationToken for long-running operations.
- ✓ Prefer Task.WhenAll / WhenAny for concurrency.
- ✓ Handle exceptions properly.
- ✓ Don't block async code with .Result or .Wait(). ☆

Common Async Methods

Method	Purpose
Task.Delay(ms)	Waits asynchronously
Task.Run(...)	Runs CPU-bound work on thread pool
Task.WhenAll(...)	Waits for all tasks to complete
Task.WhenAny(...)	Completes when any task completes
Task.Yield()	Yields control back to caller

Key Takeaways

- ✓ Async/await makes I/O operations non-blocking.
- ✓ Await pauses without blocking the thread.
- ✓ Use the right tools: Task, ValueTask, CancellationToken.
- ✓ Modern C# features improve productivity and code quality.
- ✓ Write clean, fast, and scalable code! ☆

@codedsoul_05

15. C# Interview + Real World Concepts

Note :
This chapter connects C# knowledge to interview questions and real-world scenarios you'll face in projects and products.

Why This Matters?

- ✓ Interviewers test concepts + practical problem solving.
- ✓ Real-world apps use C# features in smart ways.
- ✓ Write better, scalable, and maintainable code.

Top C# Interview Topics

Core Concepts	OOP, Collections, Generics, Delegates, LINQ, Exception Handling
Asynchronous Programming	async/await, Task, Parallelism, CancellationToken
Memory Management	GC, IDisposable, using, finalizers
Threading	Thread, ThreadPool, lock, Monitor
Performance	Struct vs Class, Boxing, String Immutability, Memory & CPU
Design & Best Practices	SOLID, DRY, KISS, Clean Code

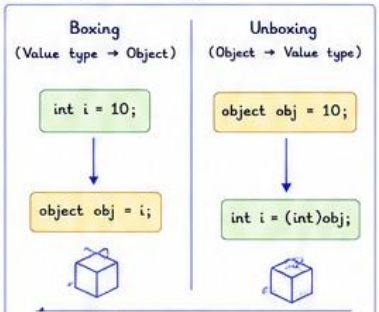
Frequently Asked Interview Questions

1. What is the difference between == and Equals() in C#?
2. What is boxing and unboxing?
3. How does garbage collection work?
4. What is the difference between List<T> and Array?
5. What is a delegate? How is it different from an interface?
6. What is the use of async and await?
7. What is the difference between class and struct?
8. How does LINQ work internally?
9. What are extension methods?
10. What is the difference between IEnumerable<T> and IQueryable<T>?

Quick Concept Reminders

Concept	In Short
Delegates	Type-safe function pointer. Used in events, callbacks.
Events	Built on delegates. Used for publish-subscribe pattern.
LINQ	Query data from collections, DB, XML in a clean way.
Async / Await	Makes I/O operations non-blocking and scalable.
Exception Handling	Keeps app stable by handling runtime errors.
Generics	Write reusable, type-safe code.
Properties	Encapsulate fields with controlled access.

Boxing vs Unboxing



Boxing has performance cost.
Avoid in performance critical code.

Real World Concept 1
Logging in Applications

Logging helps track what happens in an application.

```
using Microsoft.Extensions.Logging;

public class OrderService
{
    private readonly ILogger<OrderService> _logger;
    public OrderService(ILogger<OrderService> logger)
    {
        _logger = logger;
    }

    public void PlaceOrder(int id)
    {
        _logger.LogInformation($"Placing order {id}");
        // business Logic
        _logger.LogInformation($"Order {id} placed successfully");
    }
}
```

GC (Garbage Collection) in C#

- C# uses Automatic Memory Management.
- CLR manages memory using Garbage Collector.
- GC collects unused objects and frees memory.
- Types of Generations: 0, 1, 2
  (2 = Least frequent, most expensive to collect)

Best Practices

- ✓ Use 'using' or 'Dispose()' for unmanaged resources.
- ✓ Avoid memory leaks (unuubscribe from events).
- ✓ Avoid creating too many short-lived objects in loops.

Real World Concept 2
Repository Pattern (Data Access Layer)

Separates data access logic from business logic.

```
public interface IRepository<T>
{
    Task<T?> GetByIdAsync(int id);
    Task<IEnumerable<T>> GetAllAsync();
    Task AddAsync(T entity);
    Task UpdateAsync(T entity);
    Task DeleteAsync(int id);
}

// Makes code testable & maintainable
```

Real World Concept 3
Configuration in Applications

Store settings in appsettings.json

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=.;Database=MyDb;Trusted_Connection=True;"
  },
  "AppSettings": {
    "AppName": "CodedSoulApp",
    "PageSize": 20
  }
}
```

Read in C#

```
var conn = builder.Configuration.GetConnectionString("DefaultConnection");
var pageSize = builder.Configuration["AppSettings:PageSize"];
```

Async Best Practices

- ✓ Use async all the way.
- ✓ Avoid async void (except event handlers).
- ✓ Use ConfigureAwait(false) in library code.
- ✓ Use CancellationToken for long running operations.
- ✓ Don't block async code with .Result or .Wait().

Real World Concept 4
Handling Exceptions (Global)

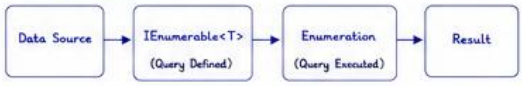
Use Middleware (ASP.NET Core)

```
app.UseExceptionHandler("/error");

app.Map("/error", (IExceptionHandlerPathFeature feature) =>
{
    var ex = feature.Error;
    // log exception
    return Results.Problem("Something went wrong!");
});
```

LINQ Internal Flow (Simplified)

LINQ uses IEnumerable<T>. Methods are executed lazily (Deferred Execution).



Code Quality & Best Practices

SOLID Principles	DRY	KISS	Immutability	Meaningful Names	Unit Testing
Write code that is maintainable and extensible.	Don't Repeat Yourself.	Keep It Simple, Stupid.	Prefer immutable objects for thread safety.	Use clear names for classes, methods, variables.	Test small parts of code in isolation.

Struct vs Class

Feature	Struct	Class
Type	Value Type	Reference Type
Stored	Stack	Heap
Default	Zero-initialized	null
Inheritance	No	Yes
Use When	Small, Immutable data	Complex, large data

Performance Tips

- ✓ Use StringBuilder for string concatenation in loops.
- ✓ Avoid boxing/unboxing.
- ✓ Use pooled resources where possible.
- ✓ Profile before optimizing.

Real World Mini Project Idea (Console App)

Build a simple Library Management System

- Add / Remove / Search Books
- Store data in file (JSON or CSV)
- Log all actions
- Handle exceptions globally
- Use async file operations
- Use Repository pattern

Sample: Async File Read

```
public static async Task<List<Book>> ReadBooksAsync(string path)
{
    if (!File.Exists(path)) return new List<Book>();

    using FileStream fs = new FileStream(path, FileMode.Open,
                                         FileAccess.Read, FileShare.Read);
    using StreamReader sr = new StreamReader(fs);
    string json = await sr.ReadToEndAsync();
    return JsonSerializer.Deserialize<List<Book>>(json) ?? new List<Book>();
}
```

Non-blocking I/O = better performance & scalability

Key Takeaways

- ✓ Understand core C# concepts deeply.
- ✓ Write clean, testable, and maintainable code.
- ✓ Use modern C# features to be productive.
- ✓ Follow best practices used in real-world projects.
- ✓ Always think from an interview + real-world perspective.